
0. Konwencje ogólne

0.1. Prefix, auth, format

- Wszystko pod prefixem: `/api`
- Format: JSON
- Autoryzacja: JWT w nagłówku. Szczegółowy opis procesu znajduje się w dokumencie [Authentication.md](#).

```
Authorization: Bearer <token>
```

- Role z bazy: `READER`, `ADMIN` (enum `user_role`).
- Daty: ISO 8601, np. `"2025-11-30T15:00:00Z"` (DateTime) lub `"2025-11-30"` (LocalDate)

0.2. Paginacja (Spring Data)

Standardowy wrapper dla list:

```
{
  "content": [],
  "pageable": {
    "pageNumber": 0,
    "pageSize": 20
  },
  "totalElements": 125,
  "totalPages": 7,
  "last": false,
  "size": 20,
  "number": 0,
  "numberOfElements": 20,
  "first": true,
  "empty": false
}
```

Query params: `?page=0&size=20&sort=title,asc`

1. Wspólne DTO i Schematy

1.1. UserDto

```
{
  "id": 1,
  "email": "reader@example.com",
  "firstName": "Jan",
  "lastName": "Kowalski",
}
```

```
"role": "READER",
"status": "ACTIVE",
"blockedReason": null,
"blockedUntil": null,
"createdAt": "2025-11-30T12:00:00Z"
}
```

1.2. BookDto

```
{
  "id": 100,
  "title": "Clean Code",
  "description": "A Handbook of Agile Software Craftsmanship.",
  "publicationYear": 2008,
  "isbn": "9780132350884",
  "category": { "id": 1, "name": "IT", "parentId": null },
  "authors": [ { "id": 5, "firstName": "Robert C.", "lastName": "Martin" } ],
  "isActive": true,
  "totalCopies": 5,
  "availableCopies": 2
}
```

1.3. LoanDto

```
{
  "id": 2001,
  "user": { "id": 1, "firstName": "Jan", "lastName": "Kowalski" },
  "bookCopy": {
    "id": 501,
    "inventoryCode": "INV-0001",
    "book": { "id": 100, "title": "Clean Code", "authors": [...] }
  },
  "loanDate": "2025-11-20T10:00:00Z",
  "dueDate": "2025-12-20T10:00:00Z",
  "returnDate": null,
  "status": "ACTIVE",
  "extensionsCount": 1
}
```

1.4. PenaltyDto

```
{
  "id": 4001,
  "user": { "id": 1, "firstName": "Jan", "lastName": "Kowalski" },
  "loanId": 2001,
  "amount": 15.0,
}
```

```
"reason": "Przekroczony termin zwrotu",
"status": "OPEN",
"createdAt": "2025-11-30T12:00:00Z",
"resolvedAt": null
}
```

2. Autoryzacja i Profil (/api/auth, /api/me)

2.1. POST /api/auth/register (Public)

Rejestracja nowego czytelnika. **Body:** `RegisterRequest` (email, password, firstName, lastName) **Response:** `201 Created` + `UserDto`

2.2. POST /api/auth/login (Public)

Logowanie do systemu. **Body:** `LoginRequest` (email, password) **Response:** `200 OK` + `AuthResponse` (token, user summary)

2.3. GET /api/auth/me (Auth)

Pobranie danych aktualnie zalogowanego użytkownika. **Response:** `200 OK` + `UserDto`

2.4. PATCH /api/auth/change-password (Auth)

Zmiana hasła przez użytkownika. **Body:** `ChangePasswordRequest` (currentPassword, newPassword) **Response:** `204 No Content`

2.5. PATCH /api/auth/me/profile (Auth)

Aktualizacja danych profilowych. **Body:** `UpdateUserRequest` (firstName, lastName) **Response:** `200 OK` + `UserDto`

3. Katalog Książek (/api/books)

3.1. GET /api/books (Public)

Przeglądanie i wyszukiwanie książek. **Query Params:** `title`, `author`, `categoryId`, `publicationYearFrom`, `publicationYearTo`, `availableOnly`, `activeOnly` **Response:** `200 OK` + `Page<BookDto>`

3.2. GET /api/books/{id} (Public)

Szczegóły konkretnej książki. **Response:** `200 OK` + `BookDto`

4. Wypożyczenia - Czytelnik (/api/loans, /api/me/loans)

4.1. GET /api/me/loans (Auth)

Aktualne wypożyczenia zalogowanego użytkownika. **Query Params:** `status` (opcjonalnie: ACTIVE, OVERDUE, etc.) **Response:** 200 OK + `Page<LoanDto>`

4.2. GET `/api/me/loans/history` (Auth)

Historia zakończonych wypożyczeń (RETURNED, LOST). **Response:** 200 OK + `Page<LoanDto>`

4.3. POST `/api/loans` (Auth)

Wypożyczenie książki (automatyczny wybór wolnego egzemplarza). **Body:** `CreateLoanRequest` (bookId) **Response:** 201 Created + `LoanDto`

4.4. POST `/api/loans/{loanId}/extend` (Auth)

Przedłużenie terminu zwrotu (limit 2 przedłużenia). **Body:** `ExtendLoanRequest` (additionalDays - opcjonalne, domyślnie 7) **Response:** 200 OK + `LoanDto`

4.5. POST `/api/loans/{loanId}/return` (Auth)

Zgłoszenie chęci zwrotu książki (zmienia status na `RETURN_REQUESTED`). **Response:** 200 OK + `LoanDto`

5. Administracja Użytkownikami (`/api/admin/users`) - ADMIN

5.1. GET `/api/admin/users`

Lista wszystkich użytkowników. **Response:** 200 OK + `List<Map>` (uproszczone dane)

5.2. GET `/api/admin/users/picker`

Lista użytkowników pod selektory (id, email, firstName, lastName). **Response:** 200 OK + `List<Map>`

5.3. PUT `/api/admin/users/{id}`

Aktualizacja danych użytkownika. **Body:** `UpdateUserRequest` **Response:** 200 OK + `UserDto`

5.4. PATCH `/api/admin/users/{id}/password`

Administracyjne ustawienie hasła. **Body:** `AdminSetPasswordRequest` (newPassword) **Response:** 204 No Content

5.5. DELETE `/api/admin/users/{id}`

Usunięcie (dezaktywacja) użytkownika. **Response:** 204 No Content

6. Administracja Katalogiem (`/api/admin/books`, `/api/admin/authors`) - ADMIN

6.1. GET `/api/admin/books`

Lista książek z pełnymi danymi dla admina. **Response:** 200 OK + `Page<BookDto>`

6.2. POST `/api/admin/books`

Dodanie nowej książki wraz z początkową liczbą egzemplarzy. **Body:** `AdminCreateBookRequest` (title, isbn, publicationYear, categoryId, authorIds, initialCopies) **Response:** 200 OK + `BookDto`

6.3. PUT `/api/admin/books/{id}`

Aktualizacja danych książki. **Body:** `AdminUpdateBookRequest` **Response:** 200 OK + `BookDto`

6.4. DELETE `/api/admin/books/{id}`

Dezaktywacja książki. **Response:** 204 No Content

6.5. GET `/api/admin/authors`

Lista autorów. **Response:** 200 OK + `Page<AuthorDto>`

6.6. POST `/api/admin/authors`

Dodanie autora. **Body:** `AdminAuthorRequest` (firstName, lastName) **Response:** 200 OK + `AuthorDto`

6.7. PUT `/api/admin/authors/{id}`

Aktualizacja danych autora. **Body:** `AdminAuthorRequest` **Response:** 200 OK + `AuthorDto`

7. Administracja Wypożyczeniami (`/api/admin/loans`) - ADMIN

7.1. GET `/api/admin/loans`

Lista wszystkich wypożyczeń w systemie. **Response:** 200 OK + `Page<LoanDto>`

7.2. POST `/api/admin/loans`

Ręczne utworzenie wypożyczenia dla konkretnego egzemplarza. **Query Params:** `userId`, `bookCopyId`, `dueDate` (opcjonalnie) **Response:** 201 Created + `LoanDto`

7.3. PUT `/api/admin/loans/{id}`

Ręczna edycja statusu lub dat wypożyczenia. **Query Params:** `status`, `dueDate`, `returnDate` **Response:** 200 OK + `LoanDto`

7.4. POST `/api/admin/loans/{id}/return/accept`

Potwierdzenie zwrotu książki (status `RETURNED`, egzemplarz staje się `AVAILABLE`). **Response:** 200 OK + `LoanDto`

7.5. POST `/api/admin/loans/{id}/return/reject`

Odrzucenie próśby o zwrot. **Response:** 200 OK + `LoanDto`

8. Kary (/api/admin/penalties) - ADMIN

8.1. GET /api/admin/penalties

Lista kar z filtrowaniem po statusie i użytkowniku. **Query Params:** `status`, `userId` **Response:** 200 OK + `Page<PenaltyDto>`

8.2. POST /api/admin/penalties

Ręczne nałożenie kary. **Body:** `AdminCreatePenaltyRequest` (`userId`, `loanId`, `amount`, `reason`) **Response:** 200 OK + `PenaltyDto`

8.3. POST /api/admin/penalties/{penaltyId}/paid

Oznaczenie kary jako opłaconej. **Response:** 200 OK + `PenaltyDto`

9. Statystyki i Dashboard (/api/admin/stats) - ADMIN

9.1. GET /api/admin/stats/summary

Dane podsumowujące dla dashboardu (liczba wypożyczeń, nowi użytkownicy, zaległości, popularne książki). **Query Params:** `from`, `to` (format YYYY-MM-DD) **Response:** 200 OK + `AdminSummaryDto`

9.2. GET /api/admin/stats/loans-per-day

Dane do wykresu liniowego wypożyczeń. **Query Params:** `from`, `to` **Response:** 200 OK + `List<AdminLoansPerDayDto>`

10. Inne

10.1. GET /health (Public)

Sprawdzenie dostępności API. **Response:** 200 OK (tekst "OK")